

# Bliss

## ISA & System Reference

Generated from `SPEC.md`

*A minimal 32-bit register machine, an assembler for it, and a small terminal-only operating system — each layer hand-rolled so that every byte is understood. This document is the reference for the virtual machine’s architecture: registers, memory, instructions, the privilege mechanism, the calling convention, the syscall ABI, and the BlissFS on-disk format.*

## Contents

|           |                                 |          |
|-----------|---------------------------------|----------|
| <b>1</b>  | <b>Architecture at a glance</b> | <b>2</b> |
| <b>2</b>  | <b>Registers</b>                | <b>2</b> |
| 2.1       | General-purpose file . . . . .  | 2        |
| 2.2       | Control registers . . . . .     | 3        |
| <b>3</b>  | <b>Privilege modes</b>          | <b>3</b> |
| 3.1       | Trap flow — ECALL . . . . .     | 3        |
| 3.2       | Return flow — ERET . . . . .    | 3        |
| <b>4</b>  | <b>Memory map</b>               | <b>4</b> |
| <b>5</b>  | <b>Instruction set</b>          | <b>4</b> |
| 5.1       | Data movement . . . . .         | 4        |
| 5.2       | Arithmetic and logic . . . . .  | 4        |
| 5.3       | Control flow . . . . .          | 5        |
| 5.4       | Privilege . . . . .             | 5        |
| <b>6</b>  | <b>Instruction formats</b>      | <b>5</b> |
| <b>7</b>  | <b>Opcode table</b>             | <b>6</b> |
| <b>8</b>  | <b>Pseudo-instructions</b>      | <b>7</b> |
| <b>9</b>  | <b>Calling convention</b>       | <b>7</b> |
| 9.1       | Register roles . . . . .        | 8        |
| 9.2       | Stack . . . . .                 | 8        |
| 9.3       | Call sequence . . . . .         | 8        |
| <b>10</b> | <b>Syscall ABI</b>              | <b>8</b> |
| 10.1      | Cause codes . . . . .           | 9        |
| 10.2      | Syscall table . . . . .         | 9        |

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>11 BlissFS — the filesystem</b>                  | <b>9</b>  |
| 11.1 Parameters . . . . .                           | 9         |
| 11.2 Disk layout . . . . .                          | 9         |
| 11.3 Superblock (block 0, first 20 bytes) . . . . . | 10        |
| 11.4 Inode (64 bytes) . . . . .                     | 10        |
| 11.5 Directory entry (28 bytes) . . . . .           | 10        |
| <b>12 Worked examples</b>                           | <b>10</b> |
| 12.1 Hello, world (bare metal, no OS) . . . . .     | 10        |
| 12.2 A user-mode program . . . . .                  | 11        |

## 1 Architecture at a glance

- 32-bit, register-based virtual machine.
- 16 general-purpose 32-bit registers (**r0–r15**).
- Fixed-width 32-bit instructions; the opcode is always bits [31:26].
- Load/store architecture: arithmetic operates on registers only; memory is touched exclusively through explicit loads and stores.
- Memory-mapped I/O — there are no dedicated I/O instructions.
- Flat 32-bit address space.
- Comparison-in-branch: there is no flags register; branch instructions take two source registers and compare them directly.

The ISA draws on RISC-V and early ARM: fixed-width instructions, a load/store core, a generous register file, branch instructions that fold in the comparison, and MMIO for every device.

## 2 Registers

### 2.1 General-purpose file

All sixteen registers are 32 bits wide. Three carry an architected role:

| Register      | Role                                                                                                                                    |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <b>r0</b>     | Assembler scratch / syscall number / return value. Reserved: never assume it survives a pseudo-instruction or a call.                   |
| <b>r1–r12</b> | Ordinary general-purpose registers (see the calling convention for how they split into argument, caller-saved and callee-saved groups). |
| <b>r13</b>    | Stack pointer (SP).                                                                                                                     |
| <b>r14</b>    | Link register (LR).                                                                                                                     |
| <b>r15</b>    | Program counter (PC). Not directly writable.                                                                                            |

## 2.2 Control registers

Four control registers sit alongside the general-purpose file and are accessed only with **MFCR** / **MTCR**.

| Name         | Index | Description                                          |
|--------------|-------|------------------------------------------------------|
| <b>tvec</b>  | 0     | Trap vector: address of the supervisor trap handler. |
| <b>epc</b>   | 1     | Exception PC: return address saved on trap.          |
| <b>cause</b> | 2     | Trap cause code (0 = syscall, ...).                  |
| <b>mode</b>  | 3     | Current privilege level (0 = user, 1 = supervisor).  |

## 3 Privilege modes

Bliss has two privilege levels: **user** (0) and **supervisor** (1). The current level lives in the **mode** control register. *The CPU boots in supervisor mode.*

### 3.1 Trap flow — ECALL

When **ECALL** executes:

1. **epc**  $\leftarrow$  **PC** — the address of the instruction following **ECALL**.
2. **cause**  $\leftarrow$  0 — syscall.
3. **mode**  $\leftarrow$  supervisor.
4. **PC**  $\leftarrow$  **tvec**.

### 3.2 Return flow — ERET

When **ERET** executes:

1. **mode**  $\leftarrow$  **user**.
2. **PC**  $\leftarrow$  **epc**.

Neither **ECALL** nor **ERET** saves or restores any general-purpose register. Preserving state across a trap is entirely the trap handler's responsibility.

## 4 Memory map

| Address range               | Use                                                |
|-----------------------------|----------------------------------------------------|
| 0x0000_0000–<br>0x0000_0FFF | Kernel code and data                               |
| 0x0000_1000–<br>0x0000_11FF | Kernel scratch buffer (disk I/O, one sector)       |
| 0x0000_1200–<br>0x0000_1FFB | Kernel stack (grows downward; SP starts at 0x1FFC) |
| 0x0000_2000–                | User program                                       |
| 0xFFFF_0000                 | Serial TX — write byte → stdout                    |
| 0xFFFF_0004                 | Serial RX — read byte ← stdin (blocks until ready) |
| 0xFFFF_0010                 | Disk sector register                               |
| 0xFFFF_0014                 | Disk buffer register                               |
| 0xFFFF_0018                 | Disk command (0 = read sector)                     |

Memory protection is not implemented. User code may read or write any address, including MMIO registers and kernel memory. This is a deliberate simplification for now.

## 5 Instruction set

### 5.1 Data movement

|            |                                                        |
|------------|--------------------------------------------------------|
| MOV        | Register to register.                                  |
| LDI16      | Load 16-bit immediate (zero-extended).                 |
| LDI32L     | Load immediate into the lower 16 bits of a register.   |
| LDI32H     | Load immediate into the upper 16 bits of a register.   |
| LDM8/16/32 | Load from memory into a register (8/16 zero-extended). |
| STR8/16/32 | Store a register to memory.                            |

### 5.2 Arithmetic and logic

|                   |                             |
|-------------------|-----------------------------|
| ADD, SUB, MUL     | Integer arithmetic.         |
| AND, OR, XOR, NOT | Bitwise logic.              |
| SLL, SLR          | Logical left / right shift. |
| SAR               | Arithmetic right shift.     |

### 5.3 Control flow

|            |                                                                                          |
|------------|------------------------------------------------------------------------------------------|
| BEQ, BNE   | Branch if equal / not equal.                                                             |
| BLT, BGE   | Branch if less-than / greater-or-equal (signed).                                         |
| BLTU, BGEU | Branch if less-than / greater-or-equal (unsigned).                                       |
| BAL        | Branch and link: PC-relative jump, save return address to a register.                    |
| BALR       | Branch and link to register: jump to an address held in a register, save return address. |
| HLT        | Halt.                                                                                    |
| NOP        | No operation.                                                                            |

### 5.4 Privilege

|               |                                                               |
|---------------|---------------------------------------------------------------|
| ECALL         | Trap from user mode to supervisor.                            |
| ERET          | Return from the trap handler to user mode.                    |
| MFCR rd, <cr> | Move from a control register into a general-purpose register. |
| MTCR <cr>, rs | Move to a control register from a general-purpose register.   |

## 6 Instruction formats

Every instruction is 32 bits wide. The opcode occupies bits [31:26] in all formats.

### R — register operations



Used by: ADD, SUB, MUL, AND, OR, XOR, SLL, SLR, SAR, MOV, NOT, BALR. For BALR, **rd** is the link destination, **rs1** is the branch target, and **rs2** is unused.

### I — two registers + immediate



Loads: **r1** = destination, **r2** = base, **imm** = offset. Stores: **r1** = source, **r2** = base, **imm** = offset. Branches: **r1**, **r2** = the two operands to compare, **imm** = signed PC-relative offset ( $\pm 512$  KB reach).

### U — one register + large immediate



LDI16 / LDI32L / LDI32H use a 16-bit immediate (upper bits unused). BAL uses the full 22-bit signed PC-relative offset ( $\pm 8$  MB reach).

**C — control register access**

|        |       |       |        |   |
|--------|-------|-------|--------|---|
| 31     | 25    | 21    | 17     | 0 |
| opcode | rd/cr | cr/rs | unused |   |

MFCR: **rd** = destination GPR, **cr** = control register index 0–3. MTCR: **cr** = control register index 0–3, **rs** = source GPR.

**N — no operands**

|        |        |   |
|--------|--------|---|
| 31     | 25     | 0 |
| opcode | unused |   |

Used by HLT, NOP, ECALL, ERET.

**7 Opcode table**

| Dec | Hex  | Mnemonic | Format |
|-----|------|----------|--------|
| 0   | 0x00 | HLT      | N      |
| 1   | 0x01 | NOP      | N      |
| 2   | 0x02 | MOV      | R      |
| 3   | 0x03 | LDI16    | U      |
| 4   | 0x04 | LDI32L   | U      |
| 5   | 0x05 | LDI32H   | U      |
| 6   | 0x06 | LDM8     | I      |
| 7   | 0x07 | LDM16    | I      |
| 8   | 0x08 | LDM32    | I      |
| 9   | 0x09 | STR8     | I      |
| 10  | 0x0A | STR16    | I      |
| 11  | 0x0B | STR32    | I      |
| 12  | 0x0C | ADD      | R      |
| 13  | 0x0D | SUB      | R      |
| 14  | 0x0E | MUL      | R      |
| 15  | 0x0F | AND      | R      |
| 16  | 0x10 | OR       | R      |
| 17  | 0x11 | XOR      | R      |
| 18  | 0x12 | NOT      | R      |
| 19  | 0x13 | SLL      | R      |
| 20  | 0x14 | SLR      | R      |
| 21  | 0x15 | SAR      | R      |
| 22  | 0x16 | BEQ      | I      |
| 23  | 0x17 | BNE      | I      |
| 24  | 0x18 | BLT      | I      |
| 25  | 0x19 | BGE      | I      |

| Dec | Hex  | Mnemonic | Format |
|-----|------|----------|--------|
| 26  | 0x1A | BLTU     | I      |
| 27  | 0x1B | BGEU     | I      |
| 28  | 0x1C | BAL      | U      |
| 29  | 0x1D | ECALL    | N      |
| 30  | 0x1E | ERET     | N      |
| 31  | 0x1F | MFCR     | C      |
| 32  | 0x20 | MTCR     | C      |
| 33  | 0x21 | BALR     | R      |

Opcodes 34–63 are reserved.

## 8 Pseudo-instructions

Pseudo-instructions are expanded by the assembler into one or more real instructions.

Most of these use **r0** as a temporary and clobber it — do not hold a live value in **r0** across a PUSH, POP, RET, or ADDI. CALL instead overwrites **r14** (the link register) with the return address.

| Pseudo       | Expands to                                        | Clob. |
|--------------|---------------------------------------------------|-------|
| LI rd, imm   | LDI32L rd, imm[15:0] · LDI32H rd, imm[31:16]      | —     |
| PUSH rs      | LDI16 r0, 4 · SUB r13, r13, r0 · STR32 rs, [r13]  | r0    |
| POP rd       | LDM32 rd, [r13] · LDI16 r0, 4 · ADD r13, r13, r0  | r0    |
| CALL label   | BAL r14, label                                    | r14   |
| RET          | BALR r0, r14                                      | r0    |
| JMP label    | BAL r0, label                                     | r0    |
| JMP rs       | BALR r0, rs                                       | r0    |
| ADDI rd, imm | LDI16 r0, imm · ADD rd, rd, r0 (imm fits 16 bits) | r0    |

CALL is a single BAL **r14**, label: it sets **r14** to the return address and jumps. RET is a single BALR **r0**, **r14**: it jumps to whatever **r14** holds. There is no implicit stack traffic — saving **r14** is the callee’s job (see below).

## 9 Calling convention

## 9.1 Register roles

| Register      | Role                             | Saved by         |
|---------------|----------------------------------|------------------|
| <b>r0</b>     | Assembler scratch / return value | — (never saved)  |
| <b>r1–r3</b>  | Argument registers               | Caller           |
| <b>r4–r7</b>  | Caller-saved temporaries         | Caller           |
| <b>r8–r12</b> | Callee-saved                     | Callee           |
| <b>r13</b>    | Stack pointer (SP)               | Always preserved |
| <b>r14</b>    | Link register (LR)               | Caller           |
| <b>r15</b>    | Program counter (PC)             | — (not writable) |

**Caller-saved** registers (**r0–r7**, **r14**) may be overwritten freely by any function; a caller that needs them after a call must spill them first. **Callee-saved** registers (**r8–r12**) must be preserved: a function that touches one pushes it on entry and pops it before returning.

## 9.2 Stack

- Full descending: SP points to the last pushed value and grows toward lower addresses.
- SP is initialised to 0x1FFC at kernel boot.
- All pushes and pops are 4-byte aligned.

## 9.3 Call sequence

**Leaf functions** — those that make no further calls — need no prologue or epilogue. **r14** is left untouched, so RET works directly:

```
bar:
    ; ... body ...
    ret                ; r14 still holds the caller's return address
```

**Non-leaf functions** must save **r14** before their own CALL clobbers it, and restore it before returning. Callee-saved registers (**r8–r12**) follow the same push-on-entry / pop-before-return pattern:

```
foo:
    push lr            ; save caller's return address (lr = r14)
    push r8            ; save any callee-saved registers used

    ; ... body, including calls to other functions ...
    call bar

    pop r8             ; restore in reverse order
    pop lr
    ret                ; jump to the restored return address
```

# 10 Syscall ABI

User programs trap into the kernel with ECALL.

- **r0** — syscall number on entry to the handler.

- **r1, r2, r3** — arguments.
- **r0** — return value, set by the handler before **ERET**.

### 10.1 Cause codes

| cause | Meaning                         |
|-------|---------------------------------|
| 0     | ECALL — syscall from user mode. |

### 10.2 Syscall table

| r0 | Name  | r1      | r2         | Returns                                 |
|----|-------|---------|------------|-----------------------------------------|
| 1  | write | buf ptr | byte count | —                                       |
| 2  | read  | —       | —          | r0 = byte read (blocks until available) |

## 11 BlissFS — the filesystem

A minimal custom filesystem for the Bliss storage device.

### 11.1 Parameters

| Property      | Value                       |
|---------------|-----------------------------|
| Block size    | 512 bytes (one sector)      |
| Magic number  | 0xB2155F2D                  |
| Max inodes    | 32                          |
| Max filename  | 24 bytes (null-padded)      |
| Max file size | $8 \times 512 = 4096$ bytes |

### 11.2 Disk layout

| Block | Contents                                               |
|-------|--------------------------------------------------------|
| 0     | Superblock                                             |
| 1–4   | Inode table (32 inodes $\times$ 64 bytes = 2048 bytes) |
| 5     | Free-block bitmap                                      |
| 6+    | Data blocks                                            |

### 11.3 Superblock (block 0, first 20 bytes)

| Offset | Size | Field                |
|--------|------|----------------------|
| 0      | 4    | Magic (0xB2155F2D)   |
| 4      | 4    | Block size (512)     |
| 8      | 4    | Inode count (32)     |
| 12     | 4    | Data start block (6) |
| 16     | 4    | Free block count     |

### 11.4 Inode (64 bytes)

| Offset | Size | Field                                        |
|--------|------|----------------------------------------------|
| 0      | 4    | File size in bytes                           |
| 4      | 1    | Type: 0 = unused, 1 = file, 2 = directory    |
| 5      | 3    | Reserved                                     |
| 8      | 32   | 8× 4-byte direct block pointers (0 = unused) |
| 40     | 24   | Reserved                                     |

Inode 0 is reserved (null). Inode 1 is the root directory.

### 11.5 Directory entry (28 bytes)

| Offset | Size | Field                         |
|--------|------|-------------------------------|
| 0      | 24   | Filename, null-padded         |
| 24     | 4    | Inode number (0 = empty slot) |

18 directory entries fit per block ( $18 \times 28 = 504$  bytes; 8 bytes unused).

## 12 Worked examples

### 12.1 Hello, world (bare metal, no OS)

Writes a string to the serial TX port one byte at a time.

```
; Hello World using .str directive and a print loop
; To run: cargo run -p asm hello.asm && cargo run hello.bin

li r0, 0xFFFF0000    ; r0 = TX address
li r1, msg           ; r1 = pointer to string
li r2, 0             ; r2 = zero (null terminator sentinel)

loop:
  ldm8 r3, [r1]       ; load next byte
  beq r3, r2, done    ; if null, stop
  str8 r3, [r0]       ; write to TX
  ldi16 r4, 1
  add r1, r1, r4      ; advance pointer
  beq r2, r2, loop    ; unconditional branch (0 == 0 always)
```

```
done:
    hlt

msg:
    .str "Hello, World!\n"
```

## 12.2 A user-mode program

Loaded by the kernel at 0x2000, prints via the `write` syscall.

```
; user.asm
.org 0x2000

start:
    ldi16 r0, 1           ; r0 = 1 (write)
    li r1, msg            ; r1 = pointer to string
    li r2, 22             ; r2 = byte count
    ecall
    hlt

msg:
    .str "Hello from user mode!\n"
```